

API

◆ 1. What is an API?

English Definition:

API (Application Programming Interface) is a set of rules that allows one software or system to interact with another.

Hinglish Explanation:

API ek **bridge** hai jo **do applications ke beech communication** karwata hai.
Jaise JavaScript ka code kisi server se data maangta hai – uske liye API ka use hota hai.

◆ 2. Types of APIs in JavaScript

✓ A. Browser APIs

(Ye APIs browser khud provide karta hai.)

API Name	Use (Kaam)
DOM API	HTML elements ko access/change karna
Console API	Console me message show karna
Fetch API	Web se data lena (HTTP request)
Geolocation API	User ka location lena
LocalStorage API	Browser me data store karna

✓ B. Web APIs

(Yeh external websites/services ki APIs hoti hain.)

Jaise:

- Weather API
- News API
- Jokes API
- Currency API

🔗 STEP 1: API kya hoti hai?

Simple Example:

- Tum Zomato app kholta hai 🍽️
- Tum "Burger" search karta hai
- Zomato ke server se data aata hai:

"Burger King", "McDonald's", "Burger Singh"...

Yaani:

- 📱 **Tumhara phone (ya app)** → Server ko request bhejta hai
- ☐ **Server** → Tere phone ko response bhejta hai

Ye jo request aur response ka system hai — usko bolte hain **API**.

🔊 **API = Communication system between two apps**

🔗 STEP 2: fetch() kya karta hai?

JavaScript me `fetch()` function ka kaam hota hai:

"Bhai, is address (URL) pe jaa, aur jo data aaye usko le aa."

```
fetch('https://api.agify.io?name=anu')
```

◆ 2. .then() kya karta hai?

```
.then(res => res.json())
```

Yeh bolta hai:

"Jab data aa jaaye, tab ye kaam karna!"

- `res` = response (jo server se aaya)
- `.json()` = us response ko JavaScript object me badlo

Yeh line ka matlab:

📁 “Server se data aaya? Ok, ab usko samajhne layak bana lo.”

◆ 3. Dusra `.then(data => ...)` kya karta hai?

```
.then(data => {  
    console.log(data);  
})
```

Code 1 →

API => `https://api.agify.io?name=anu`



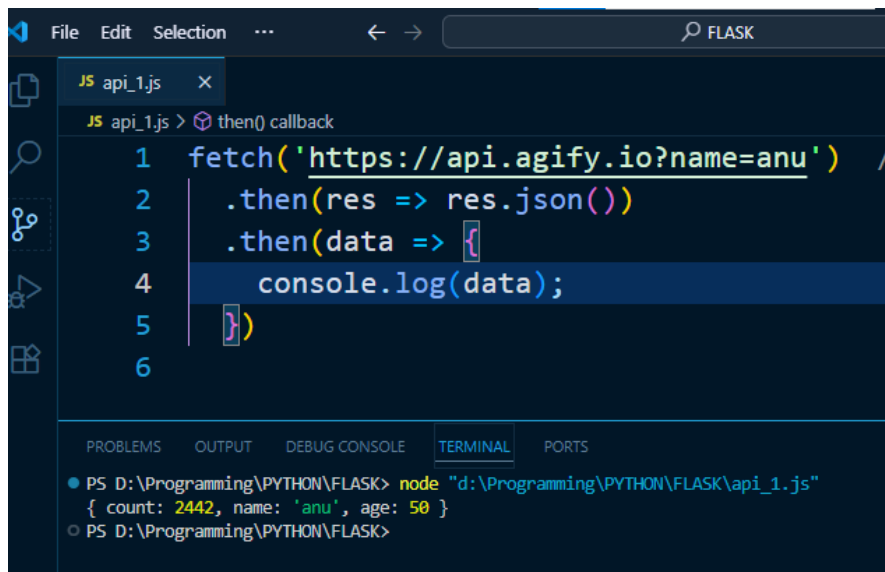
The screenshot shows a VS Code editor with a file named `api_1.js`. The code in the file is as follows:

```
1 fetch('https://api.agify.io?name=anu')  
2   .then(a => a.json())  
3   .then(b => {  
4     console.log(b);  
5   })  
6
```

The terminal at the bottom shows the command `node "d:\Programming\PYTHON\FLASK\api_1.js"` being executed, resulting in the output: `{ count: 2442, name: 'anu', age: 50 }`.

Code → [Click me](#)

CODE →2



```
JS api_1.js x
JS api_1.js > then() callback
1 fetch('https://api.agify.io?name=anu') /
2   .then(res => res.json())
3   .then(data => {
4     console.log(data);
5   })
6

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\Programming\PYTHON\FLASK> node "d:\Programming\PYTHON\FLASK\api_1.js"
{ count: 2442, name: 'anu', age: 50 }
○ PS D:\Programming\PYTHON\FLASK>
```

Code → [Click me](#)

◆ 4. .catch() kya karta hai?

```
.catch(err => {
  console.error('Error aaya!', err);
});
```

Yeh hota hai “**error pakadne wala**”.

- Internet nahi chala
- API down ho gayi
- URL galat diya

EXAMPLE → 1

Wrong URL → `https://api.agify.io000?name=anu`

Right URL → `https://api.agify.io?name=anu`

```
File Edit Selection ... < > FLASK
JS api_1.js x
JS api_1.js > then() callback
1 fetch('https://api.agify.io000?name=anu')
2   .then(res => res.json())
3   .then(data => {
4     console.log(data);
5   })
6   .catch(err => {
7     console.error('Error aaya!', err);
8   });

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
}
• PS D:\Programming\PYTHON\FLASK> node "d:\Programming\PYTHON\FLASK\api_1.js"
Error aaya! TypeError: fetch failed
    at node:internal/deps/undici/undici:13510:13
    at process.processTicksAndRejections (node:internal/process/task_queues:105:5) {
  [cause]: Error: getaddrinfo ENOTFOUND api.agify.io000
    at GetAddrInfoReqWrap.onlookupall [as oncomplete] (node:dns:122:26) {
    errno: -3008,
    code: 'ENOTFOUND',
    syscall: 'getaddrinfo',
    hostname: 'api.agify.io000'
  }
}
PS D:\Programming\PYTHON\FLASK> 
```

Code → [click me](#)

without .JSON () →

line No.2 →

🔧 Simple Definition ():

.json() ek JavaScript function hai jo API ke response ko JSON string se ek proper JS object me convert karta hai, jisse hum data.name, data.age jaise access kar sakte hain.

Matlab: Server se jawab to aaya hai, par wo raw form me hai — abhi tere use layak nahi bana.

```
File Edit Selection ... < -> FLASK
JS api_1.js JS api_2.js x
JS api_2.js > then() callback
1 fetch("https://api.agify.io?name=anu")
2 // .then(a=>a.json())
3 .then(res => {
4   console.log(res);
5 });

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

'content-length': '54',
connection: 'keep-alive',
vary: 'accept-encoding',
'content-encoding': 'gzip',
'cache-control': 'max-age=0, private, must-revalidate',
'x-request-id': 'GGKoQJDfRExhR6ZDqHOC',
'access-control-allow-credentials': 'true',
'access-control-allow-origin': '*',
'access-control-expose-headers': 'x-rate-limit-limit,x-rate-limit-remaining,x-rate-li
'x-rate-limit-limit': '100',
'x-rate-limit-remaining': '80',
'x-rate-limit-reset': '51401'
},
body: ReadableStream { locked: false, state: 'readable', supportsBYOB: true },
bodyUsed: false,
ok: true,
redirected: false,
type: 'basic',
url: 'https://api.agify.io/?name=anu'
}
PS D:\Programming\PYTHON\FLASK>
```

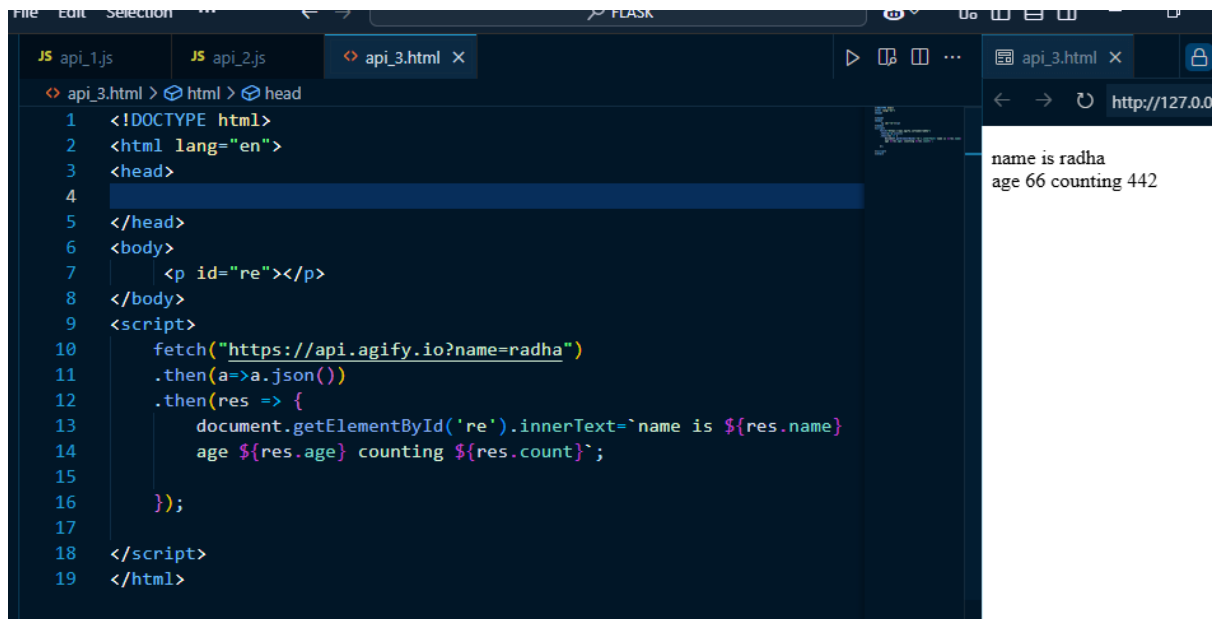
With JS →

```
File Edit Selection ... < -> FLASK
JS api_1.js JS api_2.js x
JS api_2.js > then() callback
1 fetch("https://api.agify.io?name=anu")
2   .then(a=>a.json())
3   .then(res => {
4     console.log(res);
5   });
6

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● PS D:\Programming\PYTHON\FLASK> node "d:\Programming\PYTHON\FLASK\api_2.js"
{ count: 2442, name: 'anu', age: 50 }
○ PS D:\Programming\PYTHON\FLASK>
```

USING HTML →



The screenshot shows a web browser window with a single tab titled 'api_3.html'. The address bar shows the URL 'http://127.0.0.1:5500/api_3.html'. The page content displays the result of a JavaScript fetch request: 'name is radha' and 'age 66 counting 442'. The browser's developer tools are open, showing the 'api_3.html' file in the editor. The code is as follows:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4
5 </head>
6 <body>
7   <p id="re"></p>
8 </body>
9 <script>
10   fetch("https://api.agify.io?name=radha")
11     .then(a=>a.json())
12     .then(res => {
13       document.getElementById('re').innerText=`name is ${res.name}
14       age ${res.age} counting ${res.count}`;
15     });
16 </script>
17 </html>
```

Code → [Click me](#)

RANDOM {JOCK CODE(API)} →

```
JS api_2.js × JS api_1.js JS api_3.js api_3.html code_1.html ×
code_1.html > html > body
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Simple API Example</title>
5 </head>
6 <body>
7
8 <h1>Random Joke from API</h1>
9 <button onclick="getJoke()">Get Joke</button>
10
11 <p id="joke"></p>
12
13 <script>
14   function getJoke() {
15     fetch('https://official-joke-api.appspot.com/random_joke')
16       .then(response => response.json())
17       .then(data => {
18
19         document.getElementById('joke').innerText = `${data.setup}`;
20       })
21       .catch(error => {
22         document.getElementById('joke').innerText = "Error fetching joke.";
23       });
24   });
25 }
26 </script>
27
28 </body>
29 </html>
30
```

Click me → [CODE](#)

Wikipedia Info Fetch (API) →

API =>

'https://en.wikipedia.org/api/rest_v1/page/summary/Salman_Khan'

Code 1 →

The screenshot shows a web browser window with the URL `http://127.0.0.1:3000/code_2`. The browser displays a page titled "Simple API Example" with a button labeled "Click". Below the button, the text "Wikipedia Data Fetch" is visible. The code editor on the left shows the following HTML and JavaScript code:

```
1 <html>
2 <head>
3 <title>Simple API Example</title>
4 </head>
5 <body>
6
7 <h1>Wikipedia Data Fetch </h1>
8 <button onclick="getJoke()">Click </button>
9 <p id="joke"></p>
10
11 <script>
12 function getJoke() {
13
14
15
16   fetch('https://en.wikipedia.org/api/rest_v1/page/summary/Salman_Khan')
17
18   .then(response => response.json())
19   .then(data => {
20
21     document.getElementById('joke').innerText = data.extract;
22   })
23   .catch(error => {
24     document.getElementById('joke').innerText = "Error fetching joke.";
25   });
26 }
27 </script>
28
29 </body>
30 </html>
```

Code → [click me](#)

Code -2 →

User i/p Code

code_2.html

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Simple API Example</title>
5 </head>
6 <body>
7
8 <h1>Wikipedia Data Fetch</h1>
9 <input type="text" id="ko">
10
11 <button onclick="getJoke()">Click</button>
12
13 <p id="joke"></p>
14
15 <script>
16   function getJoke() {
17     let c=document.getElementById('ko').value;
18
19     fetch(`https://en.wikipedia.org/api/rest_v1/page/summary/${c}`)
20
21     .then(response => response.json())
22     .then(data => {
23
24       document.getElementById('joke').innerText = data.extract || "no";
25     })
26     .catch(error => {
27       document.getElementById('joke').innerText = "Error fetching joke.";
28     });
29   });
30 }
31 </script>
32
33 </body>
34 </html>
35
```

Simple API Example

http://127.0.0.1:3000

Wikipedia Data Fetch

Salman Salim Khan is an Indian actor, film producer, and television personality who predominantly works in Hindi films. In a career spanning over three decades, his awards include two National Film Awards as a film producer, and two Filmfare Awards as an actor. He has been cited in the media as one of the most popular and commercially successful actors of Indian cinema. Forbes included him in listings of the highest-paid celebrities in the world, in 2015 and 2018. He has starred in the annual highest-grossing Hindi films of 10 individual years, the highest for any actor.

Please reopen the preview.

Code → [click](#)

Feature / Point	● Without Flask (Only HTML + CSS + JS)	□ With Flask (HTML + CSS + JS + Flask)
📁 File Type	.html, .css, .js	.py (Flask), .html (Jinja), .css, .js
⚙️ Functionality	Static website (fixed content)	Dynamic website (content from Python backend)
□ Logic Handling	Only frontend logic (like UI interaction)	Frontend + Backend logic (form, data processing, API)
⬇️ Form Handling	Only basic client-side validation	Real form submission + server-side validation
🗄️ Database Support	✗ No direct database support	✓ Full database integration (SQLite, MySQL, etc.)
∞ API Handling	Only frontend can call APIs using <code>fetch()</code>	Flask can create and consume APIs
🔄 Data Flow	One-way (static pages)	Two-way (Client ↔ Server communication)
📁 Use Case	Portfolios, landing pages, static blogs	Login systems, dashboards, e-commerce, dynamic apps
💡 Template Engine	✗ Not available	✓ Jinja2 (for reusable HTML code)
🔒 Security Features	✗ None (client-side only)	✓ Server-side validation, session, authentication
📦 Project Level	Basic / Beginner Projects	Intermediate to Advanced Projects

 Profit / Benefit	Skill in frontend design, UI, interaction	Full-stack web development skills, better job opportunities	
 Hosting Requirement	Can be hosted on GitHub Pages easily	Needs a server (like Heroku, PythonAnywhere, Render)	
☐ Testing	Only UI testing	Full unit testing possible (Flask Testing)	